INTRODUCTION

➢The word algorithm comes from the name of the person author -Abu Jafar Mohammed Ibn Musa Al khowarizmi who wrote
A text book entitled-"Algorithmi de numero indorum" Now term" Algorithmi "in the title of the book led to the term Algorithm.

➢An algorithm is an effective method for finding out the solution for a given problem. It is a sequence of instruction
That conveys the method to address a problem

➢**Algorithm :** Step by step procedure to solve a computational problem is called Algorithm.

or

➢An Algorithm is a step-by-step plan for a computational procedure that possibly begins with an input and yields an output value in a finite number of steps in order to solve a particular problem.



INTRODUCTION

- An algorithm is a set of steps of operations to solve a problem performing calculation, data processing, and automated reasoning tasks.

- An algorithm is an efficient method that can be expressed within finite amount of Time and space.

- The important aspects of algorithm design include creating an efficient algorithm to solve a problem in an efficient way using minimum time and space.

- To solve a problem, different approaches can be followed. Some of them can be efficient with respect to time consumption, whereas other approaches may be memory efficient.



PROPERTIES OF ALGORITHM

TO EVALUATE AN ALGORITHM WE HAVE TO SATISFY THE FOLLOWING CRITERIA:

1.INPUT: The Algorithm should be given zero or more input.

2.OUTPUT: At least one quantity is produced. For each input the algorithm produced value from specific task.

3.DEFINITENESS: Each instruction is clear and unambiguous.

4.FINITENESS: If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.

5.EFFECTIVENESS: Every instruction must very basic so that it can be carried out, in principle, by a person using only pencil & paper.



ALGORITHM (CONTD…)

- A well-defined computational procedure that takes some value, or set of values, as *input* and produces some value, or set of values, as *output.*

- Written in a pseudo code which can be implemented in the language of programmer's choice.

PSEUDO CODE: A notation resembling a simplified programming language, used in program design.



How To Write an Algorithm

Step-1:start
Step-2:Read a,b,c
Step-3:if a>b
if a>c
print a is largest
else
if b>c
print b is largest
else
print c is largest
Step-4 : stop

Step-1: start
Step-2: Read a,b,c
Step-3:if a>b then go to step 4
otherwise go to step 5
Step-4:if a>c then
print a is largest otherwise
print c is largest
Step-5: if b>c then
print b is largest otherwise
print c is largest
step-6: stop



DIFFERENCES

Algorithm	Program
1.At design phase	1.At Implementation phase
2.Natural language	2.written in any programming language
3.Person should have Domain knowledge	3.Programmer
4.Analyze	4.Testing



ALGORITHM SPECIFICATION

Algorithm can be described (Represent) in four ways.

1.Natural language like English:

When this way is chooses, care should be taken, we should ensure that each & every statement is definite.
(no ambiguity)

2. Graphic representation called flowchart:

This method will work well when the algorithm is small& simple.

3. Pseudo-code Method:

In this method, we should typically describe algorithms as program, which resembles language like Pascal & Algol(Algorithmic Language).

4.Programming Language:

we have to use programming language to write algorithms like C, C++,JAVA etc.



PSEUDO-CODE CONVENTIONS

1. Comments begin with // and continue until the end of line.

2. Blocks are indicated with matching braces { and }.

3. An identifier begins with a letter. The data types of variables are not explicitly declared.

```
node= record
        {
    data type 1 data 1;
    data type n data n;
      node *link;
        }
```

4. There are two Boolean values **TRUE** and **FALSE**.

Logical Operators

AND, OR, NOT

Relational Operators

<, <=,>,>=, =, !=



5. Assignment of values to variables is done using the assignment statement.

<Variable>:= <expression>;

6. Compound data types can be formed with records. Here is an example,

Node. Record

```
{
 data type – 1   data-1;
.
.
.
 data type – n  data – n;
  node * link;
}
```

Here link is a pointer to the record type node. Individual data items of a record can be accessed with → and period.



7. The following looping statements are employed.

For, while and repeat-until While Loop:

While < condition > do

{

<statement-1>

..

..

<statement-n>

}

For Loop:

For variable: = value-1 to value-2 step step do

{

<statement-1>

.

.

.

<statement-n>

}



repeat-until:

repeat
<statement-1>
.
.
.
<statement-n>
until<condition>

8. A conditional statement has the following forms.

→ If <condition> then <statement>
→ If <condition> then <statement-1>
Else <statement-1>



Case statement:

Case
{
: <condition-1> : <statement-1>
.
.
.
: <condition-n> : <statement-n>
: else : <statement-n+1>
}

9. Input and output are done using the instructions read & write. No format is used to specify the size of input or output quantities



Contd…

10. There is only one type of procedure: Algorithm, the heading takes the form,

Algorithm Name (Parameter lists)

consider an example, the following algorithm fields & returns the maximum of n given numbers:

```
1.  algorithm Max(A,n)
2.  // A is an array of size n
3.  {
4.  Result := A[1];
5.  for i:= 2 to n do
6.  if A[i] > Result then
7.  Result :=A[i];
8.  return Result;
9.  }
```



Issue in the study of algorithm

1. **How to create an algorithm.**
2. **How to validate an algorithm.**
3. **How to analyses an algorithm**
4. **How to test a program.**

1 .How to create an algorithm: To create an algorithm we have following design technique

a) Divide & Conquer
b) Greedy method
c) Dynamic Programming
d) Branch & Bound
e) Backtracking



2.How to validate an algorithm: Once an algorithm is created it is necessary to show that it computes the correct output for all possible legal input , this process is called algorithm validation.

3.How to analyses an algorithm: Analysis of an algorithm or performance analysis refers to task of determining how much computing Time & storage algorithms required.

a) Computing time-Time complexity: Frequency or Step count method

b) Storage space- To calculate space complexity we have to use number of input used in algorithms.

4.How to test the program: Program is nothing but an expression for the algorithm using any programming language. To test a program we need following

a) Debugging: It is processes of executing programs on sample data sets to determine whether faulty results occur & if so correct them.

b) Profiling or performance measurement is the process of executing a correct program on data set and measuring the time & space it takes to compute the result.



ANALYSIS OF ALGORITHM

PRIORI	POSTERIORI
1.Done priori to run algorithm specific system	1.Analysis after running on a it on system.
2.Hardware independent	2.Dependent on hardware
3.Approximate analysis	3.Actual statistics of an algorithm
4.Dependent on no of time do posteriori statements are executed	4.They do not analysis

PERFORMANCE ANALYSIS

Performance Analysis: An algorithm is said to be efficient and fast if it take less time to execute and consumes less memory space at run time is called Performance Analysis.

1. SPACE COMPLEXITY:

The space complexity of an algorithm is the amount of Memory Space required by an algorithm during course of execution is called space complexity .There are three types of space

a) Instruction space :executable program
b) Data space: Required to store all the constant and variable data space.
c) Environment: It is required to store environment information needed to resume the suspended space.

2. TIME COMPLEXITY:

The time complexity of an algorithm is the total amount of time required by an algorithm to complete its execution.



SPACE COMPLEXITY

Now there are two types of space complexity

a) Constant space complexity

b) Linear(variable)space complexity



1. Space Complexity

The space complexity of an algorithm is the amount of memory it needs to run to complete.

Space needed by an algorithm is given by

S(P)=C(fixed part)+Sp(Variable part)

fixed part: independent of instance characteristics.

eg: space for simple variables, constants etc

Variable part: Space for variables whose size is dependent on particular problem instance.



2. Time Complexity:

The time complexity of an algorithm is the amount of computer time it needs to run to complete.

T(P)= compile time+ execution time

T(P)= tp (execution time)

Step count:

- For algorithm heading→0
- For Braces→0
- For expressions→1
- for any looping statements→no.of times the loop is repeating.

DAA

1.Constant time complexity : If a program required fixed amount of time for all input values is called Constant time complexity .

Example : int sum(int a , int b)

```
{
return a+b;
}
```



2. Linear time complexity: If the input values are increased then the time complexity will changes.

- comments = 0 step
- Assignment statement= 1 step
- condition statement= 1 step
- loop condition for n times = n+1 steps
- body of the loop = n steps



Example :
```
int sum(int A[],int n)
 {
int sum=0,i;
   for (i=0;i<n;i++)
   sum=sum+A[i];
        return sum;
```

cost	repetation	total
1	1	1
1+1+1	1+(n+1)+n	2n+2
2	n	2n
1	1	1
		4n+4



Algorithm-1

```
1. Algorithm abc(a,b,c)
2. {
3. return a+b*c+(a+b-c)/(a+b) +4.0;
4. }
```

→0
→0
→1
→0
---
1 unit

Algorithm-3

```
Algorithm RSum(a,n)
{
if(n≤0) then return 0.0;
else return Rsum(a,n-1)+a[n];
}
```

Algorithm-2

```
1. Algorithm sum(a,n)
2. {
3. s=0.0;
4. for i=1 to n do
5.  s= s+a[i];
6.  return s;
7. }
```

→0
→0
→1
→n+1
→n
→1
→0

2n+3

DAA

$$T(n)= \begin{cases} 2 & \text{if } n=0 \\ 2+ T(n-1) & \text{if } n>0 \end{cases}$$

$T(n)= 2+ T(n-1)$
$= 2+ (2+T(n-2))= 2*2+T(n-2)$
$=2*2+(2+T(n-3))=2*3+T(n-3)$
.
.
$=2*n+T(n-n)= 2n+T(0)$
$T(n) =2n+2$

TIME COMPLEXITY

The time T(p) taken by a program P is the sum of the compile time and the run time(execution time)

Statement	S/e	Frequency	Total
1. Algorithm Sum(a,n)	0	-	0
2.{	0	-	0
3. S=0.0;	1	1	1
4. for i=1 to n do	1	n+1	n+1
5. s=s+a[I];	1	n	n
6. return s;	1	1	1
7. }	0	-	0
Total			2n+3



ANALYSIS OF AN ALGORITHM

- The goal of analysis of an algorithm is to compare algorithm in running time and also Memory management.
- Running time of an algorithm depends on how long it takes a computer to run the lines of code of the algorithm.

Running time of an algorithm depends on

1.Speed of computer

2.Programming language

3.Compiler and translator

Examples: binary search, linear search



ANALYSIS OF AN ALGORITHM

- experimental analysis is valuable, but it has its limitations.
- If we wish to analyze a particular algorithm without performing experiments on its running time, we can take the following more analytic approach directly on the high-level code or pseüdó-codé.
- Wé dèfine à sèt of high-level primitive operations that are largely independent from the programming language used and can be identified also in the pseudo-code.
- Primitive operatioñs include the following:
  - Assigning .a value to a variable
  - Calling a method
  - Performing an arithmetic operation (for example, adding two numbers).
  - Comparing two numbers
  - Indexing, into an array
  - Following an object reference
  - Returning, from a method.



- Specifically, a primitive operation corresponds to a low-level instruction with an execution time that depends on the hardware and software environment but is, nevertheless constant
- Instead of trying to determine the specific execution time of each primitive operation, we **will simply count how many primitive operations are executed, and use this number t as a high-level estimate of the running time of the algonthm**
- This operation count will correlate to an actual running time in a specific hardware and software environment, for each primitive operation corresponds to a constant-time instruction, and there are only a fixed number of primitive operations
- The implicit assumption in this approach is that the running times of different primitive operations will be fairly similar
- **Thus, the number, t, of primitive operations an algorithm performs will.be proportional to the actual running time of that algorithm.**



RANDOM ACCESS MACHINE MODEL(RAM MODEL)

- approach of simply counting pnmitive operations gives rise to a computational model called the Random Accèss Machine (RAM)
- This model, which should not be confused with "random access nemory," views a computer simply as a CPU connected to a bank of memory cells.
- Each memòry cell stores a word, which can be a number, a character string, or an address, that is, the value of a base type.
- **The tèrm "random access" refers to the ability of the CPU tö access an arbitrary memory cell with one primitive Operation.**
- To keep the model simple, we **do not place any specific limits on the size of numbers that can be stored in words of memory.**
- We assume the CPU in the RAM model can perform any primitive operation in a constant number of steps, which do not depend on the size of the input.
- Thus, an accurate bound on the number of primitive operations an algorithm performs cörresponds directly to the running time of that algorithm in the RAM model.



KINDS OF ANALYSIS

1.Worst-case: (usually)

- T(n) = maximum time of algorithm on any input of size n.

2.Average-case: (sometimes)

- T(n) = expected time of algorithm over all inputs of size n.
- Need assumption of statistical distribution of inputs.

3.Best-case:

- T(n) = minimum time of algorithm on any input of size n.

COMPLEXITY:

Complexity refers to the rate at which the storage time grows as a function of the problem size



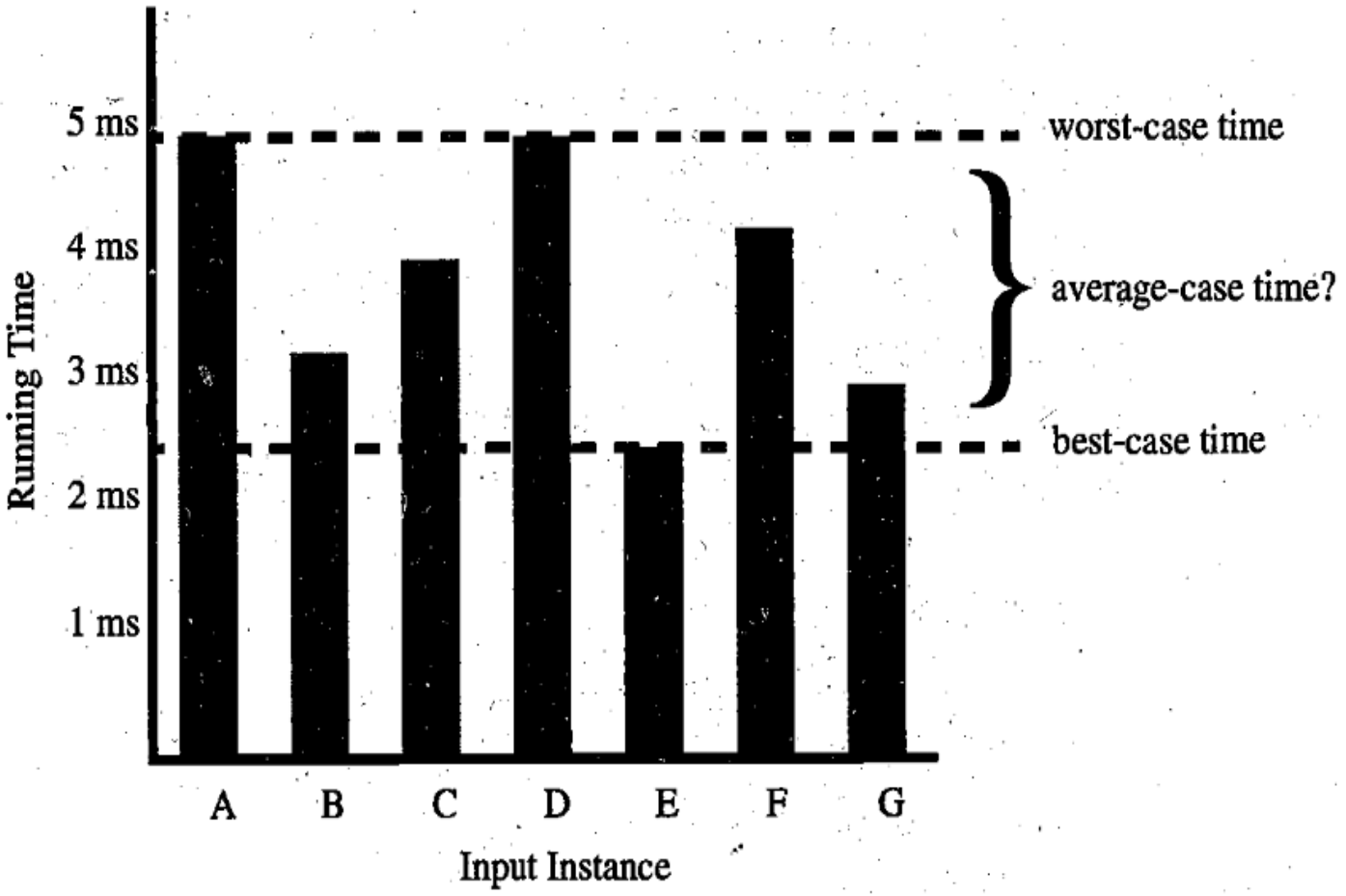


Figure 1.3: The difference between best-case and worst-case time. Each bar represents the running time of some algorithm on a different possible input.

ASYMPTOTIC ANALYSIS:

- Expressing the complexity in term of its relationship to know function. This type analysis is called asymptotic analysis.
- The main idea of Asymptotic analysis is to have a measure of efficiency of an algorithm , that doesn't depends on

1.Machine constants.

2.Doesn't require algorithm to be implemented.

3.Time taken by program to be prepare.



ASYMPTOTIC NOTATION

ASYMPTOTIC NOTATION: The mathematical way of representing the Time complexity.

The notation we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are the set of natural numbers.

Definition : It is the way to describe the behavior of functions in the limit or without bounds.

Asymptotic growth: The rate at which the function grows…

"growth rate" is the complexity of the function or the amount of resource it takes up to compute.

Growth rate ⟶ **Time +memory**



They are 3 asymptotic notations are mostly used to represent time complexity of algorithm.

1. Big oh (O)notation
2. Big omega (Ω) notation
3. Theta(Θ) notation
4. Little oh notation
5. Little omega(Ω) notation



1.Big oh (O)notation :

- Asymptotic “less than”(slower rate).
- represent upper bound of algorithm run time.
- Big oh (O calculate maximum ecution.
- worst case time complexity.



1.Big O-notation

- For a given function $g(n)$, we denote by $O(g(n))$ the set of functions

$$O(g(n)) = \left\{ \begin{array}{c} f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ s.t.} \\ 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0 \end{array} \right\}$$

- We use *O*-notation to give an asymptotic upper bound of a function, to within a constant factor.
- $f(n) = O(g(n))$ means that there existes some constant c s.t. $f(n)$ is always $\le cg(n)$ for large enough n.

- c>0,n_o >1

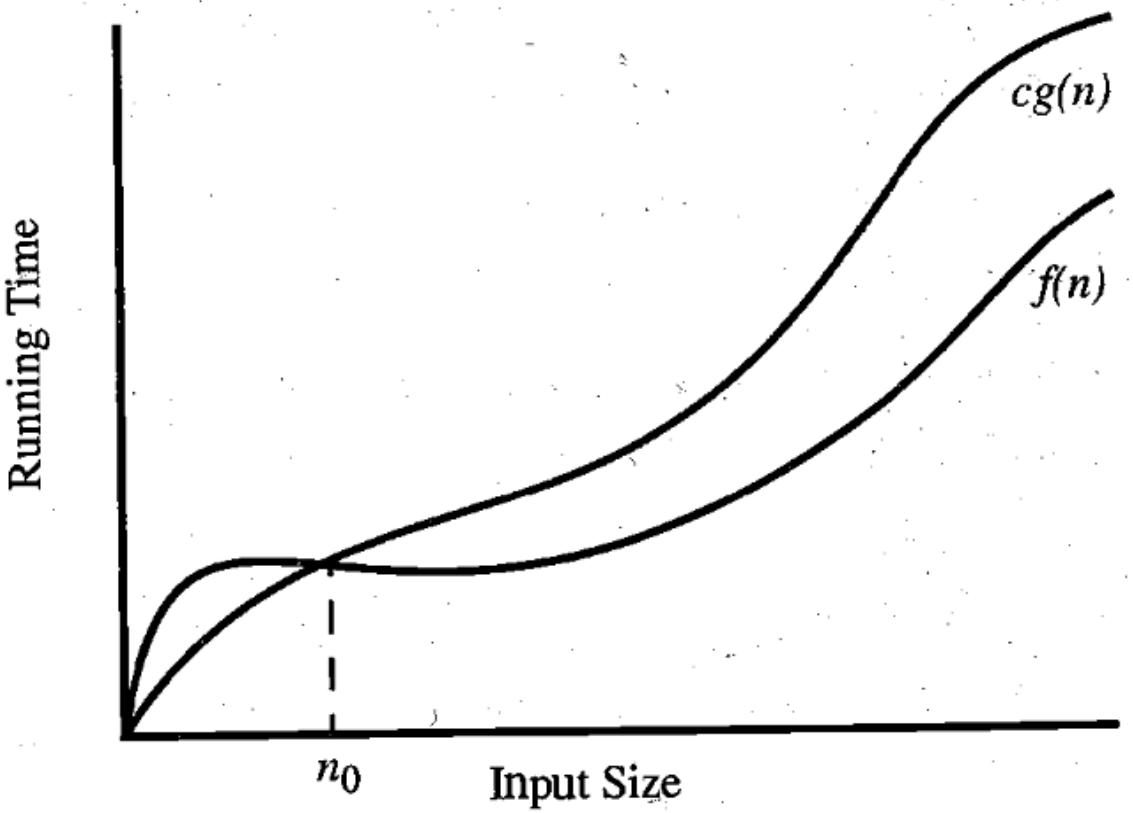


Figure 1.5: Illustrating the "big-Oh" notation. The function $f(n)$ is $O(g(n))$, for $f(n) \leq c \cdot g(n)$ when $n \geq n_0$.

EXAMPLES

Example : f(n)=3n +2 & g(n)= n

Formula : $f(n) <= c\ g(n)$ $n >= n_0$, $c>0$, $n_0 >= 1$

f(n)=2n+3 & g(n)=n

Now 3n+2<=c.n

3n+2<=4.n

Put the value of n =1

5<=4 false

N=2 8<=8 true now n0>2 For all value of n>2 & c=4

now f(n)<= c.g(n)

3n+2<=4n for all value of n>2

Above condition is satisfied this notation takes maximum amount of time to execute .so that it is called worst case complexity.



Table 2.1 shows the variation of values of the polynomial and $5n^2$.

Table 2.1 Comparison of $f(n)$ and $g(n)$

n	$4*n*n + 5*n + 3$	$5*n*n$
1	12	5
2	29	20
3	54	45
4	87	80
5	128	125
6	177	180

Hence, it becomes evident from the table that the value of n for which $5 \times n^2$ becomes greater than $4 \times n^2 + 5 \times n + 3$ is 6.

It can, therefore, be stated that

$$g(n) = 5 \times n^2, \text{ for } n \geq 6$$



2.Ω-OMEGA NOTATİON

Ω-Omega notation : Asymptotic "greater than"(faster rate).

It represent Lower bound of algorithm run time.

By using Big Omega notation we can calculate minimum amount of

time. We can say that it is best case time complexity.

Formula : $f(n)>=c\ g(n)$ $n>=n_0$, $c>0$, $n_0>=1$

where c is constant, n is function

- Lower bound
- Best case



Ω-Omega notation

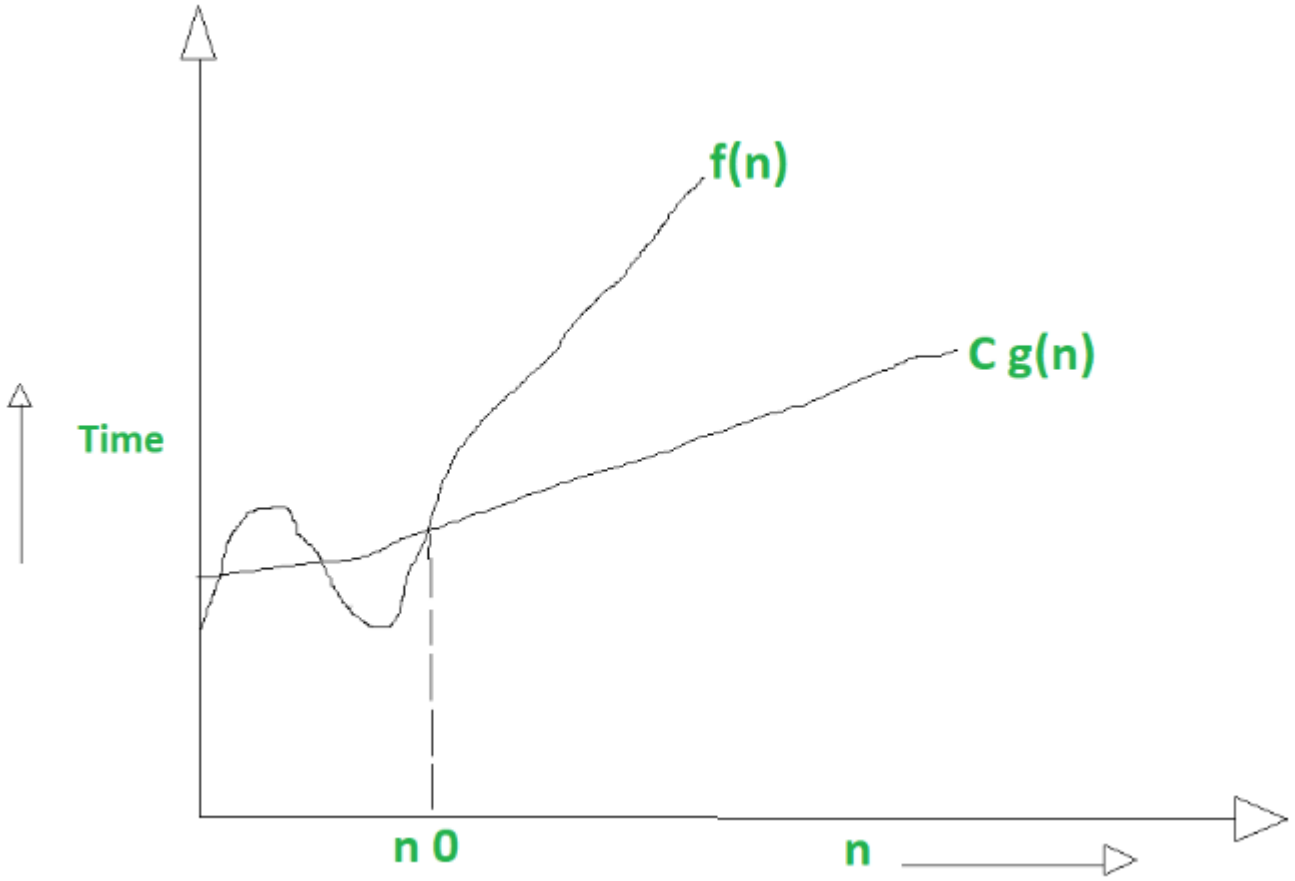
- For a given function $g(n)$, we denote by $\Omega(g(n))$ the set of functions

$$\Omega(g(n)) = \left\{ \begin{array}{c} f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ s.t.} \\ 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0 \end{array} \right\}$$

- We use Ω-notation to give an asymptotic lower bound on a function, to within a constant factor.
-

- $f(n) = \Omega(g(n))$ means that there exists some constant c s.t. $f(n)$ is always $\ge cg(n)$ for large enough n.




f(n)
C g(n)
Time
n 0
n

EXAMPLES

Example : f(n)=3n +2

Formula : f(n)>=c g(n) $n>=n_0$, c>0 , n_0 >=1

f(n)=3n+2

3n+2>=1*n, c=1 put the value of n=1

n=1 5>=1 true n0>=1 for all value of n

It means that f(n)= Ω g(n).

Table 2.2 Comparison of $f(n)$ and $g(n)$

N	$5*n*n + 2*n + 5$	$4*n*n$
1	12	4
2	29	16
3	56	36
4	93	64
5	140	100
6	197	144

polynomial and $4n^2$. Hence, it becomes evident from the table that $4 \times n^2$ is less than $5 \times n^2 + 2 \times n + 7$ for all values of $n \geq 1$. So it can be stated that $g(n) = 4 \times n^2$, for $n \geq 1$.



ϑ -THETA NOTATİON

Theta (Θ) notation : Asymptotic "Equality"(same rate).

It represent average bond of algorithm running time.

By using theta notation we can calculate average amount of time.

So it called average case time complexity of algorithm.

Formula : $c_1\ g(n) <= f(n) <= c_2\ g(n)$

where c is constant, n is function

- Average bound

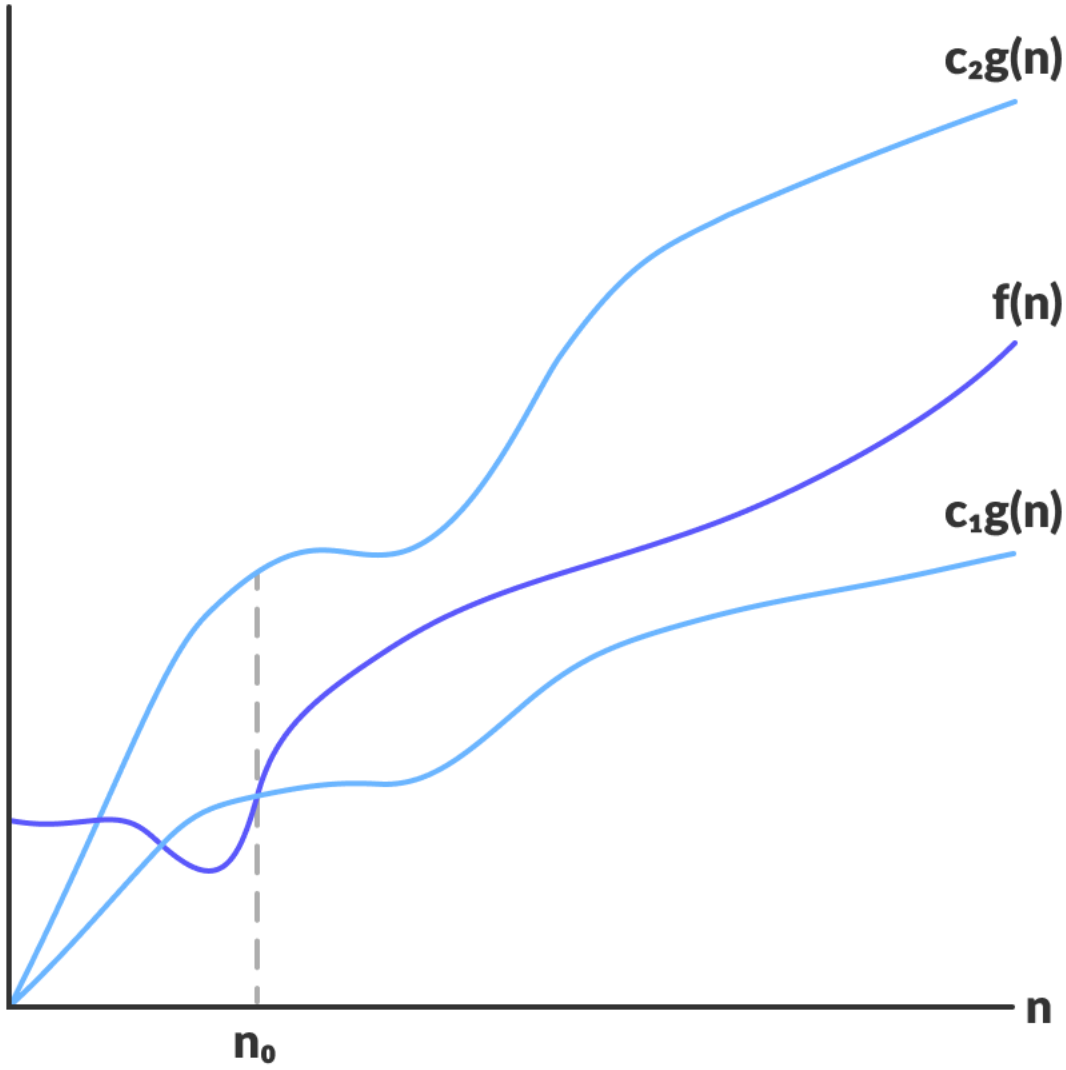
-Theta notation

- For a given function $g(n)$, we denote by $\Theta(g(n))$ the set of functions

$$\Theta(g(n)) = \left\{ \begin{array}{c} f(n) : \text{there exist positive constants } c_1, c_2, \text{and } n_0 \text{ s.t.} \\ 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0 \end{array} \right\}$$

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be "sand-wiched" between $c_1 g(n)$ and $c_2 g(n)$ or sufficiently large n.
- $f(n) = \Theta(g(n))$ means that there exists some constant c_1 and c_2 s.t. $c_1 g(n) \le f(n) \le c_2 g(n)$ for large enough n.





$f(n) = \Theta(g(n))$

EXAMPLES

Example : f(n)=3n+2

Formula : c_1 g(n)<=f(n)<=c_2 g(n)

f(n)=2n+3

1*n<=3n+2<=4*n now put the value of n=1 we get 1<=5<=4 false

n=2 we get 2<=8<=8 true

n=3 we get 3<=11<=12 true

Now all value of n>=2 it is true above condition is satisfied.



Table 2.3 shows the variation of values of the polynomial and $4n^2$ and $2 \times n^2$.

Table 2.3 Comparison of $g(n)$ and $\theta(g(n))$

n	$3*n*n + 2*n + 1$	$4*n*n$	$2*n*n$
1	6	4	2
2	17	16	8
3	34	36	18
4	57	64	32
5	86	100	50
6	121	144	72
7	162	196	98
8	209	256	128
9	262	324	162
10	321	400	200
11	386	484	242

Hence, it becomes evident from the table that the value of n for which $2 \times n^2$ is less than $3 \times n^2 + 2 \times n + 1$ is 1 and the value of n for which the function $4n^2$ becomes greater than $3 \times n^2 + 2 \times n + 1$ is 3.



Illustration 2.1 Two algorithms A_1 and A_2 run on the same machine. The running time of A_1 is $100n^2$ and the running time of A_2 is 2^n. For what value of n, A_1 runs faster than A_2?

Solution Table 2.4 shows the variation of values of $100n^2$ and 2^n with n. It may be noted that for $n \geq 15$, 2^n exceeds $100n^2$. So, till $n = 14$, A_2 runs faster.

Table 2.4 Variation of $100n^2$ and 2^n with n

n	$100n^2$	$2n$
1	100	2
2	400	4
3	900	8
4	1600	16
5	2500	32
6	3600	64
7	4900	128
8	6400	256
9	8100	512
10	10,000	1024
11	12,100	2048
12	14,400	4096
13	16,900	8192
14	19,600	16,384
15	22,500	32,768

2^n is greater than $100n^2$ for all $n \geq 15$



In order to make the concept clear, let us also analyse the graph of $100n^2$ and 2^n with n. Figure 2.5 shows the graph. It may be noted that after $n = 15$, 2^n is way ahead of $100n^2$.

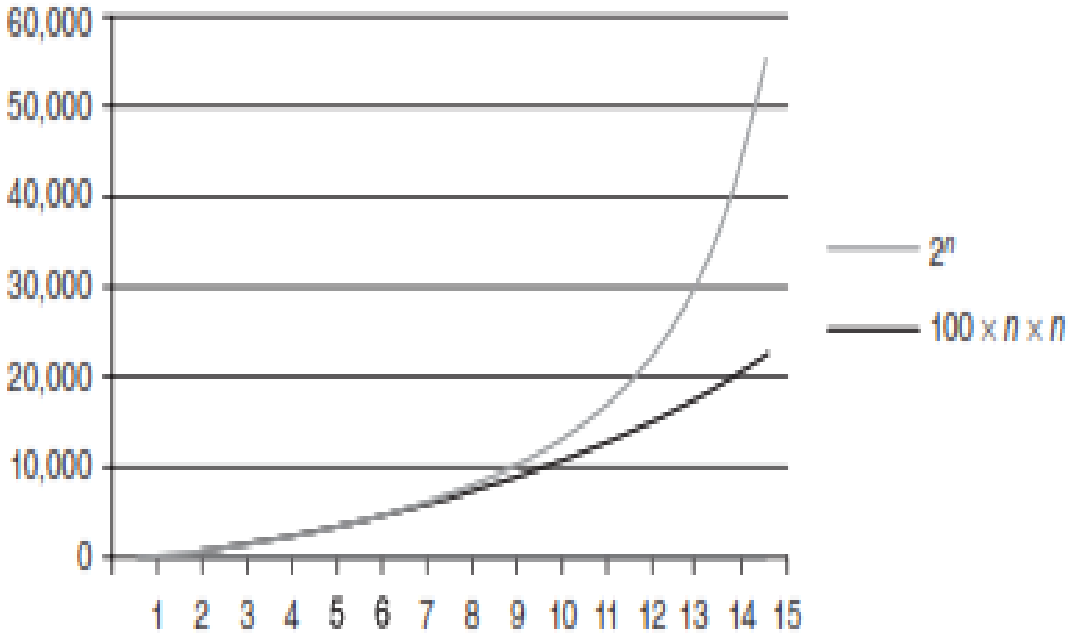


Figure 2.5 Variation of $100n^2$ and $2n$ with n

Illustration 2.2 Two algorithms A_1 and A_2 run on the same machine. The running time of A_1 is $100n^{30}$ and the running time of A_2 is 2^n. Can A_1 run faster than A_2?

Solution The question is similar to the first illustration, except for the fact that the power of n in this case is 30. Now from a quick observation, it appears that A_1 will always take more time as compared to A_2, but Fig. 2.6 shows the variation of values of $100n^{30}$ and 2^n with n. It may be noted that for $n \geq 245$, 2^n exceeds $100n^{30}$.

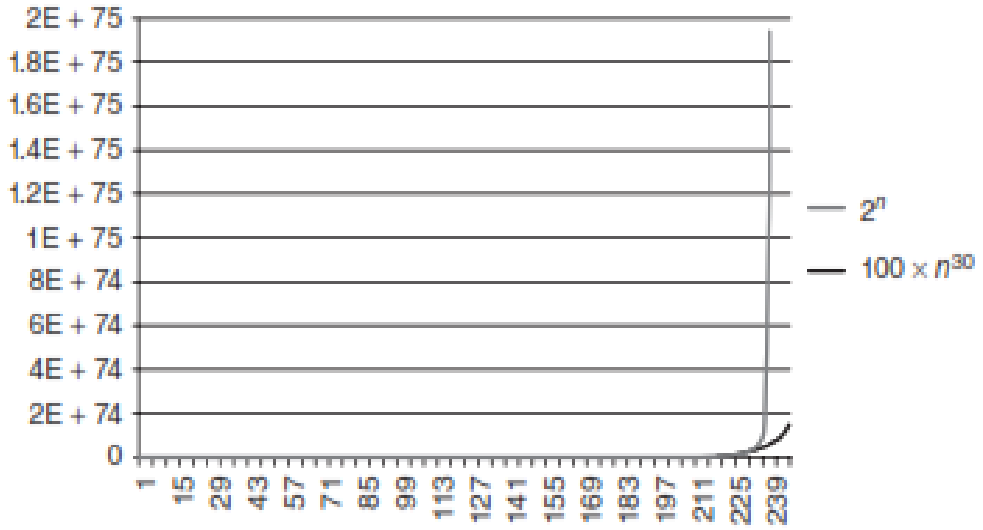


Figure 2.6 Variation of $100n^2$ and 2^n with n

So, it is worth remembering that for large values of n, 2^n will always be greater than $c \times n^m$, n being the number of inputs and m is any integer.



4.LITTLE OH NOTATION

- Little o notation is used to describe an upper bound that cannot be tight. In other words, loose upper bound of f(n).

Slower growth rate

f(n) grows slower than g(n)

- Let f(n) and g(n) are the functions that map positive real numbers. We can say that the function f(n) is o(g(n)) if for **any real positive constant c**, there exists an integer constant n0 ≤ 1 such that f(n) > 0.



- There are also some ways of saying that one function is strictly less than or strictly greater than another asymptotically, but these are not used as often as the big-Oh, big-Omega, and big-Theta.

Example 1.11: *The function* $f(n) = 12n^2 + 6n$ *is* $o(n^3)$ *and* $\omega(n)$.

Proof: *Let us first show that* $f(n)$ *is* $o(n^3)$. *Let* $c > 0$ *be any constant. If we take* $n_0 = (12+6)/c$, *then, for* $n \geq n_0$, *we have*

$$cn^3 \geq 12n^2 + 6n^2 \geq 12n^2 + 6n.$$

Thus, $f(n)$ *is* $o(n^3)$.

To show that $f(n)$ *is* $\omega(n)$, *let* $c > 0$ *again be any constant. If we take* $n_0 = c/12$, *then, for* $n \geq n_0$, *we have*

$$12n^2 + 6n \geq 12n^2 \geq cn.$$

Thus, $f(n)$ *is* $\omega(n)$. ■



- Using mathematical relation, we can say that f(n) = o(g(n)) means,

  if

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

- **Example on little o asymptotic notation:**

1. If $f(n) = n^2$ and $g(n) = n^3$ then check whether

   f(n) = o(g(n)) or not.



Sol:

$$\lim_{n\to\infty} \frac{n^2}{n^3}$$

$$= \lim_{n\to\infty} \frac{1}{n}$$

$$= \frac{1}{\infty}$$

$$= 0$$

The result is 0, and it satisfies the equation mentioned above. So we can say that f(n) = o(g(n)).



- Another asymptotic notation is little omega notation. it is denoted by (ω).
- Little omega (ω) notation is used to describe a loose lower bound of f(n).
- Faster growth rate
- F(n) grows faster than g(n)
- If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

5.LITTLE OMEGA(Ω) NOTATION

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

stated as f(n)=ωg(n)



Some functions appear often in the analysis of algorithms and data structures, and we often use special terms to refer to them. Table 1.6 shows some terms commonly used in algorithm analysis.

logarithmic	**linear**	**quadratic**	**polynomial**	**exponential**
$O(\log n)$	$O(n)$	$O(n^2)$	$O(n^k)$ $(k \geq 1)$	$O(a^n)$ $(a > 1)$

Table 1.6: Terminology for classes of functions.



EXAMPLE OF ASYMPTOTIC NOTATION

Problem:-Find upper bond ,lower bond & tight bond range for functions: f(n)= 2n+5

Solution:-Let us given that f(n)= 2n+5 , now g(n)= n

lower bond=2n, upper bond =3n, tight bond=2n

For Big –oh notation(O):- according to definition

f(n)<=cg(n) for Big oh we use upper bond so

f(n)=2n+5, g(n)=n and c=3 according to definition

2n+5<=3n

Put n=1 7<=3 false Put n=2 9<=6 false Put n=3 14<=9 false Put n=4 13<=12 false Put n=5 15<=15 true

now for all value of n>=5 above condition is satisfied. C=3 n>=5



2. Big - omega notation :- $f(n) >= c.g(n)$ we know that this

Notation is lower bond notation so $c=2$

Let $f(n)=2n+5$ & $g(n)=2.n$

Now $2n+5>=c.g(n)$;

$2n+5>=2n$ put $n=1$

We get $7>=2$ true for all value of $n>=1, c=2$ condition is satisfied.

3. Theta notation :- according to definition

$c1.g(n)<=f(n)<=c2.g$



Running Time	Maximum Problem Size (n)		
	1 second	1 minute	1 hour
$400n$	2,500	150,000	9,000,000
$20n\lceil \log n \rceil$	4,096	166,666	7,826,087
$2n^2$	707	5,477	42,426
n^4	31	88	244
2^n	19	25	31

Table 1.7: Maximum size of a problem that can be solved in one second, one minute, and one hour, for various running times measured in microseconds.



Functions Ordered by Growth Rate
$\log n$
$\log^2 n$
$\sqrt{n}$
n
$n \log n$
n^2
n^3
2^n

An algorithm is said to run in

- logarithmic time if $T(n) = O(log(n))$
- polylogarithmic time if $T(n) = O(log(n)^k)$ (also written as $T(n) = O(log^k(n))$)

That means they are the same for $k = 1$. Otherwise they are different and your other examples are all polylogarithmic. I'm not sure how exactly to explain *what* the difference is but maybe a picture will help you:

Also, as a notational shorthand, we use $\log^c n$ to denote the function $(\log n)^c$ and we use $\log \log n$ to denote $\log(\log n)$. Rather than show how we could derive

n	$\log n$	$\sqrt{n}$	n	$n \log n$	n^2	n^3	2^n
2	1	1.4	2	2	4	8	4
4	2	2	4	8	16	64	16
8	3	2.8	8	24	64	512	256
16	4	4	16	64	256	4,096	65,536
32	5	5.7	32	160	1,024	32,768	4,294,967,296
64	6	8	64	384	4,096	262,144	1.84×10^{19}
128	7	11	128	896	16,384	2,097,152	3.40×10^{38}
256	8	16	256	2,048	65,536	16,777,216	1.15×10^{77}
512	9	23	512	4,608	262,144	134,217,728	1.34×10^{154}
1,024	10	32	1,024	10,240	1,048,576	1,073,741,824	1.79×10^{308}

Table 1.10: Growth of several functions.

